

***Universitatea Ovidius din Constanta
Facultatea de Matematica si Informatica
Specializarea Informatica***

Ingineria sistemelor software

Curs 8

Conf. univ. dr. Crenguta M. Puchianu
2020



Proiectare software



Stiluri arhitecturale

Un stil arhitectural definește o familie de sisteme în termenii unui șablon de organizare structurală.

Un stil arhitectural specifică de obicei:

- un vocabular de proiectare,
- descrierea semanticii vocabularului și
- constrângeri asupra modului în care este folosit acest vocabular.

Exemple:

- Sisteme flux de date
- Sisteme call-reply
- Sisteme bazate pe evenimente
- Sisteme client-server
- Sisteme stratificate



Sisteme flux de date

- ❑ Sunt caracterizate de modul in care datele sunt transmise in sistem
- ❑ Aceste arhitecturi au doua sau mai multe componente de procesare a datelor ce transforma datele de intrare in date de iesire.
- ❑ Stilul arhitectural pipeline: este limitat la o secventa liniara de filtre



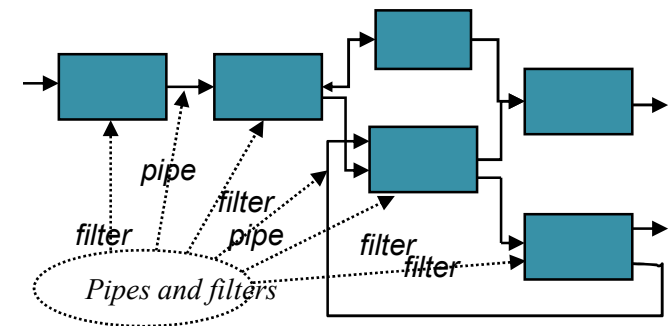
- ❑ Stilul arhitectural conducte si filtre:

Un filtru citeste fluxuri de date ca resurse de intrare si produce fluxuri de date ca resurse de iesire.

Conductele au rolul de a transmite datele de iesire furnizate de filtre la intrarile altor filtre.

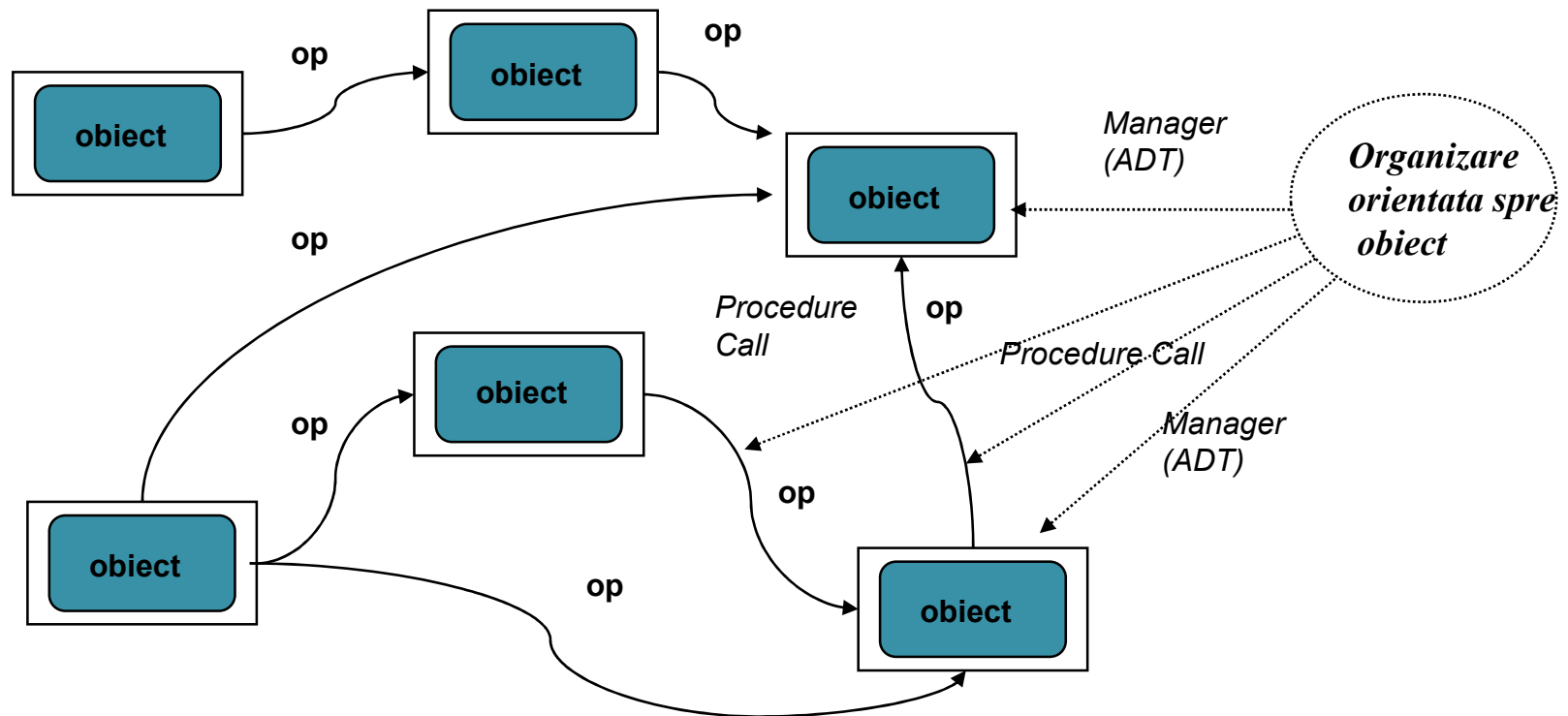
Semantica: conductele pastreaza ordinea transmiterii datelor, iar filtrele sunt invocate in mod nedeterminist

Constrangeri: sunt interzise ciclarile



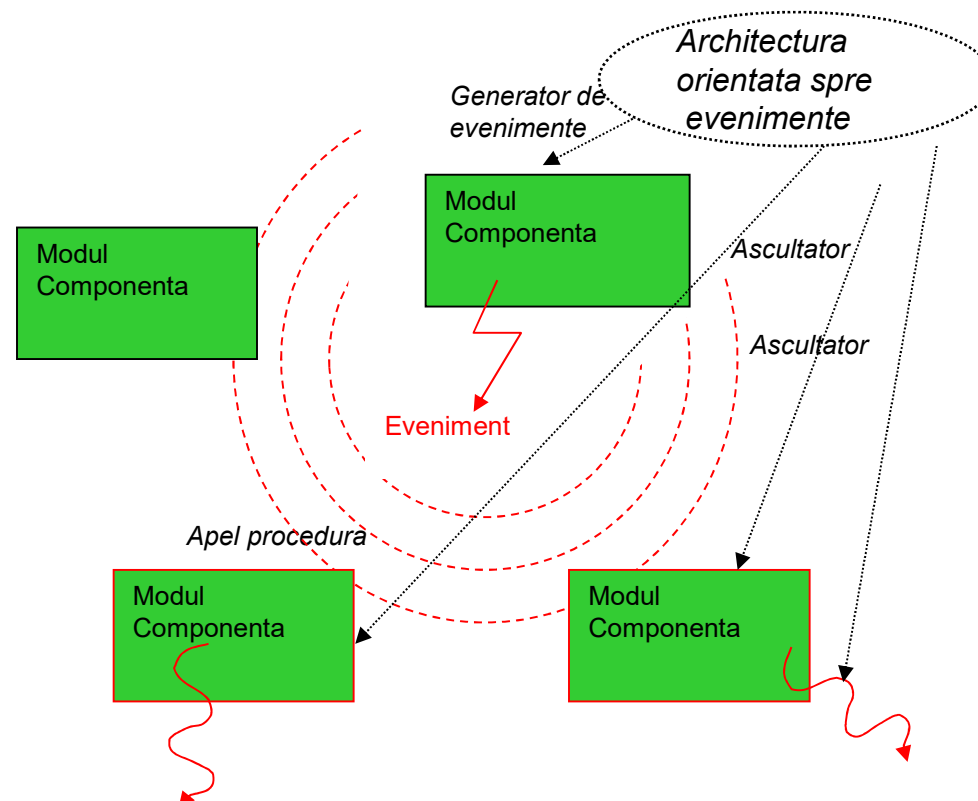
Sisteme call-reply

- ❑ Datele impreuna cu operatii sunt incapsulate in obiecte sau tipuri abstracte de date
- ❑ Reprezentarea interna a acestora este inaccesibila altor obiecte
- ❑ Obiectele comunica prin intermediul apelurilor procedurilor sau functiilor/metode



Sisteme bazate pe evenimente

- ❑ O componenta poate anunta alte componente de aparitia unui sau mai multor evenimente.
- ❑ Celelalte componente se pot inregistra la un timp de evenimente prin asocierea de proceduri sau functii.
- ❑ Cand un eveniment de acel tip este anuntat, sistemul insusi invoca procedurile, functiile sau metodele componentelor care gestioneaza evenimentul



Sisteme client-server

- ❑ Un sistem client-server este format dintr-un numar de procese client si un proces server care coopereaza pentru a realiza o activitate de elaborare a datelor.
- ❑ Server-ul este o entitate pasiva care asteapta cereri de servicii de la clienti. Un client este o entitate activa care trimite cereri de servicii catre server.
- ❑ Clientul si server-ul sunt de obicei pe calculatoare diferite, dar pot rula si pe un acelasi calculator.
- ❑ Avantaje:
 - independenta clientilor si a server-ului pentru ca acestia se cunosc in retea;
 - faciliteaza modificarea, in sensul ca modificarea unui client nu afecteaza server-ul;
 - permite controlul strict al resurselor aflate pe server.

Dezavantaj: trafic intens in retea

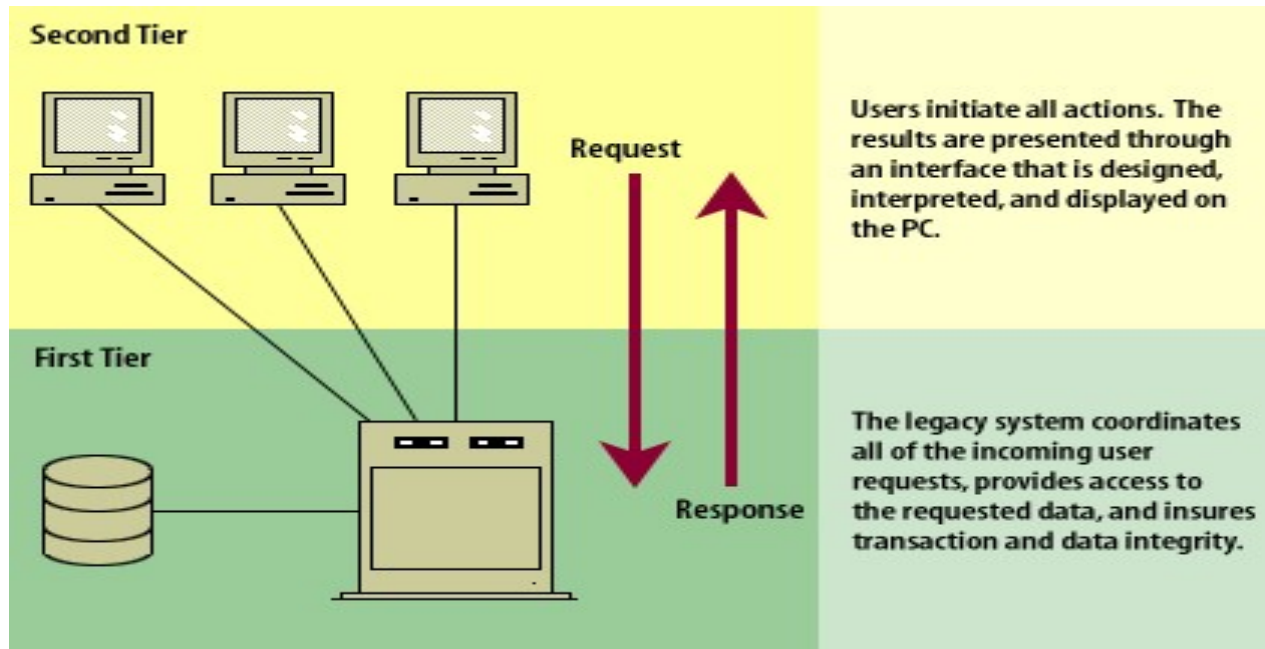


Sisteme stratificate

- ☐ Un sistem stratificat este organizat pe nivele, fiecare nivel furnizand servicii stratului superior si servind ca client pentru stratul inferior.
- ☐ Interactiunile dintre componente sunt realizate pe baza de protocoale.
- ☐ De obicei constrangerea este ca interactiunile sa aiba loc numai intre straturile adiacente. Astfel de arhitecturi se numesc inchise.
- ☐ Intr-o arhitectura deschisa, un strat poate accesa straturi inferioare, indiferent de adancime.



Arhitecturi 2-tier



Nivel 1. GUI – ferestre, panouri

Date de intrare ↓ ↑ Date de iesire

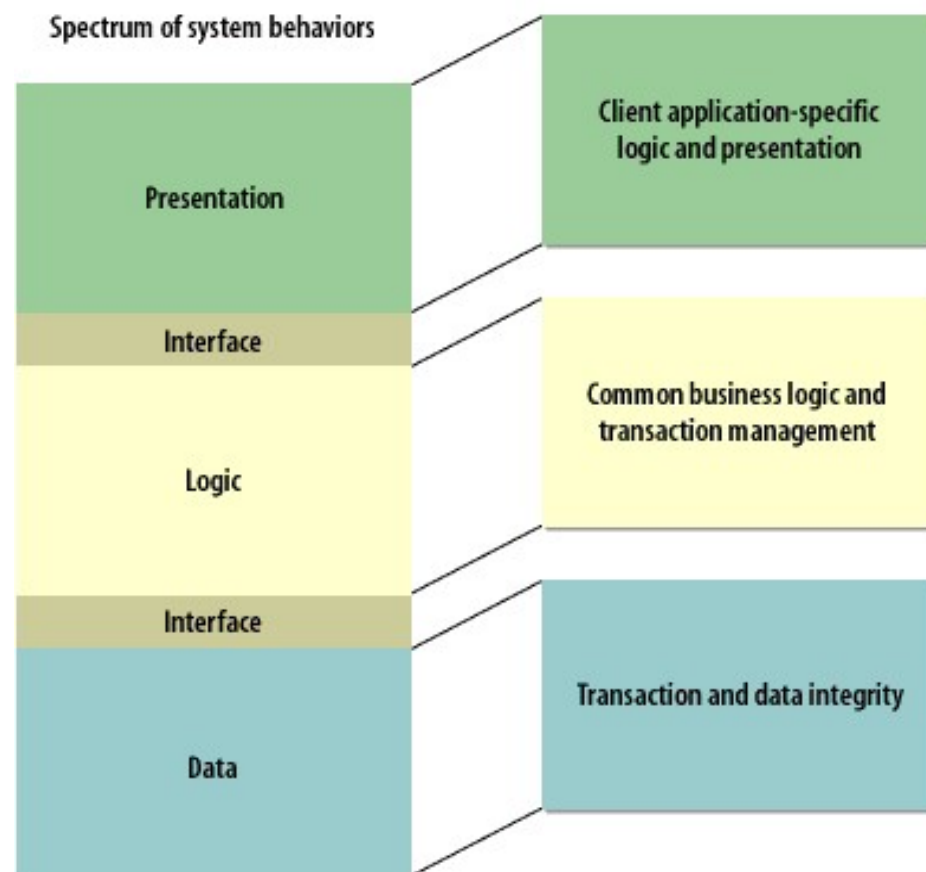
Nivel 2. Nivel persistent



Arhitecturi 3-tier

Contine urmatoarele straturi:

- *Stratul de logica prezentarii* este responsabil cu interactiunea cu utilizatorii
- *Stratul de logica domeniului* ce contine principalele functionalitati ale sistemului
- *Stratul de logica surselor de date* implementeaza comunicarea stratului superior cu baza de date, si poate contine alte aplicatii/subsisteme



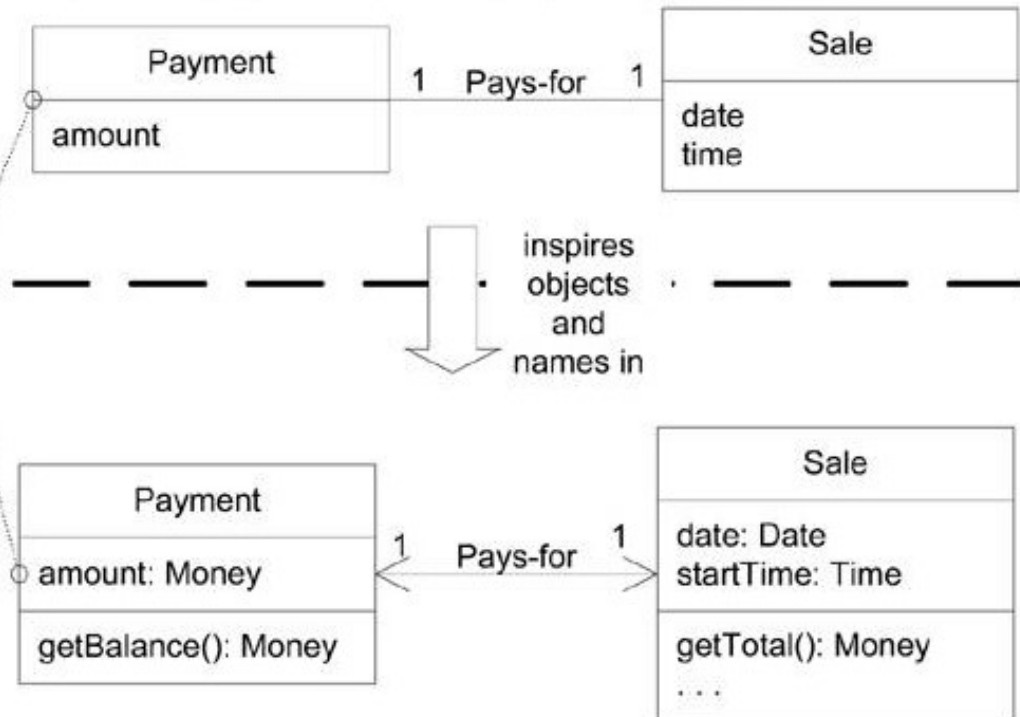
Domeniul problemei vs Stratul domeniu

A Payment in the Domain Model is a concept, but a Payment in the Design Model is a software class. They are not the same thing, but the former inspired the naming and definition of the latter.

This reduces the representational gap.

This is one of the big ideas in object technology.

UP Domain Model
Stakeholder's view of the noteworthy concepts in the domain.



Domain layer of the architecture in the UP Design Model

The object-oriented developer has taken inspiration from the real world domain in creating software classes.

Therefore, the representational gap between how stakeholders conceive the domain, and its representation in software, has been lowered.



Stilul architectural MVS

(Model-View-Separation Principle)

- Afirmă că obiectele model (domeniu) nu ar trebui să aibă cunoștințe directe despre obiectele vedere (UI).
- Clasele de domeniu încapsulează informațiile și comportamentul legate de logica aplicației.

Principiul MVS are cel puțin două părți:

1. Nu cuplați obiecte non-UI direct la obiecte UI. De ce? Deoarece ferestrele sunt legate de o anumită aplicație, în timp ce (în mod ideal) obiectele nonferestre pot fi refolosite în aplicații noi sau atașate la o nouă interfață.

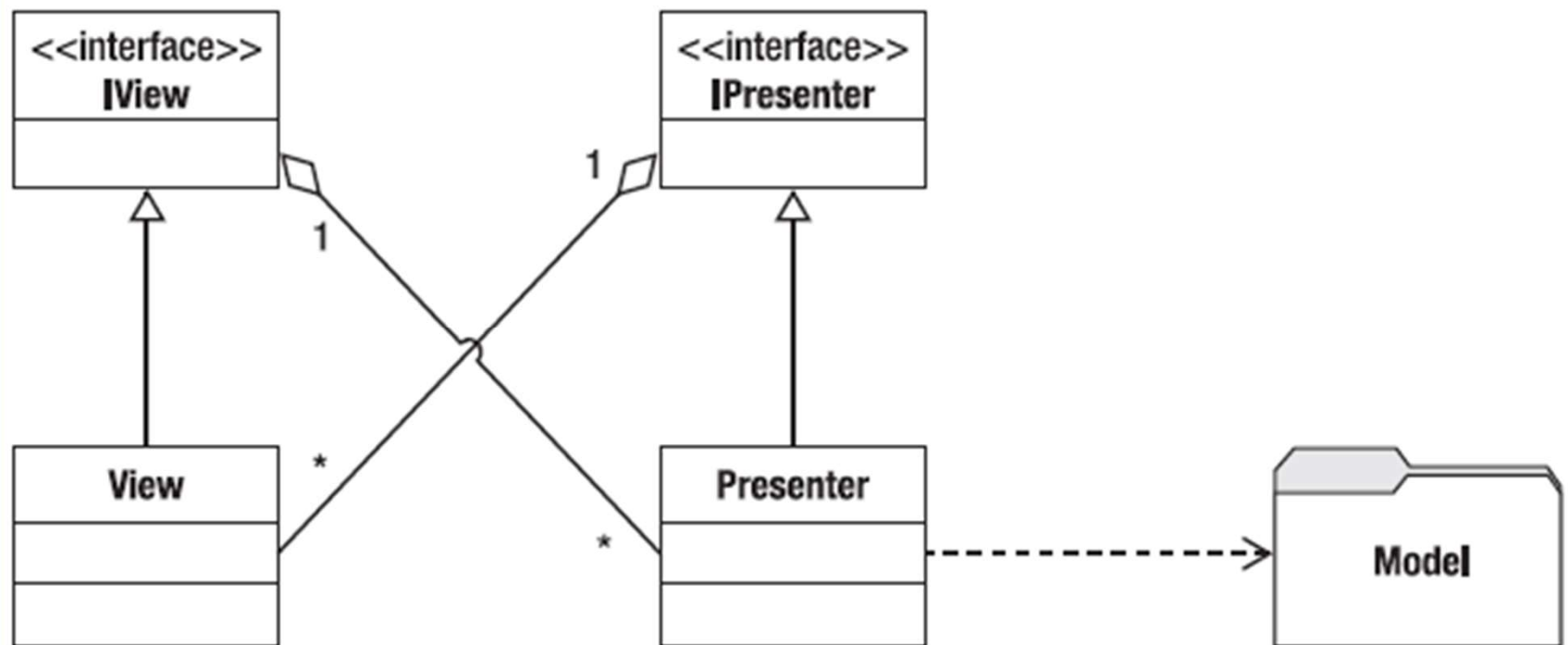
2. Nu puneți logica aplicației (cum ar fi un calcul al impozitelor) în operațiile unui obiect UI. Obiectele UI ar trebui să inițializeze doar elemente UI, să primească evenimente UI (cum ar fi un clic al mouse-ului pe un buton) și să delege cereri pentru logica aplicației către obiecte non-UI (cum ar fi obiectele de domeniu).

În acest context, Model este un sinonim pentru stratul de domeniu al obiectelor. View este un sinonim pentru obiecte UI, cum ar fi ferestre, pagini web, applet-uri și rapoarte.



Stilul architectural MVP (Model-View-Presenter)

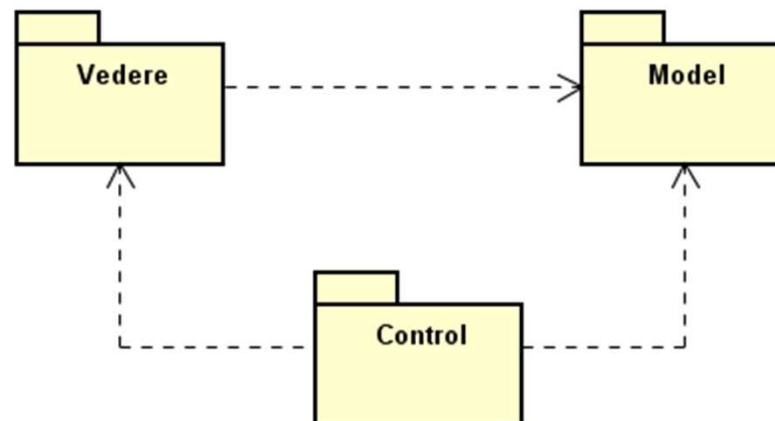
- ❑ Conform acestui stil architectural, presenter actioneaza ca un mediator intre straturile vedere si model.
- ❑ Presenter poate interoga clasele vedere pentru a obtine date, ceea ce implica o dependenta cu celelalte doua straturi.
- ❑ Este folosit in ASP.Net si Windows Forms.



Stilul architectural MVC (Model-View-Controller)

Componentele sunt in general obiecte clasificate dupa responsabilitatile lor in trei categorii:

- obiecte view (frontiera): obiecte cu care actorii software interactioneaza direct: UI sau obiecte vedere ale claselor entitate
- obiecte entitate reprezinta obiecte de domeniu ale sistemului.
- obiecte de control au responsabilitatea de a rezolva fluxul logic si de a gestiona evenimentele de sistem produse de actorii software in interactiunea lor cu obiectele de frontiera.



Stilul architectural MVVM (Model-View-ViewModel)

- ❑ View este similar cu vederea in MVC. Contine numai elemente UI.
- ❑ ViewModel: este echivalent cu Presentation Model din stilul architectural MVP
- ❑ Model formeaza logica business a aplicatiei.
- ❑ Este folosit in Microsoft Visual Studio.

